

Фінальний проєкт:

Дедлайн 23:45, 26 черв.



Що будемо робити

Твоя задача — створити RESTful API системи управління університетом, яка працює зі студентами, викладачами, курсами та зарахуваннями.

У проєкті буде реалізовано:

- CRUD-операції для всіх основних сутностей
- Фільтрація й пошук
- Зарахування на курси, оплати, оцінки
- Побудова транскриптів та обчислення GPA
- Повна документація API через Swagger
- Зберігання даних у **реальній** реляційній базі (PostgreSQL)
- Юніт-тести та інтеграційні тести

Після завершення ти матимеш робочий бекенд-застосунок рівня Trainee/Junior.

Мета завдання

Мета фінального проєкту — закріпити практичні навички створення веб-бекенду за допомогою **Spring Boot** та **Spring Data JPA**.

Ти навчишся проєктувати багатoshарову архітектуру, документувати API, писати тести та працювати з реальною базою даних.

Які навички ти прокачаєш

1. **Spring Boot 3+** — Робота з REST-контролерами, конфігурацією, DI, валідацією. 2. **Spring Data JPA + PostgreSQL** — Взаємодія з базою даних, побудова запитів, робота зі зв'язками One-to-Many, Many-to-One.

3. **REST API Design** — HTTP-методи, статус-коди, DTO, статична та динамічна фільтрація.
4. **Багатошарова архітектура** — Controller → Service → Repository → DB.
5. **Тестування (JUnit + Mockito + Spring Test)** — Юніт-тести бізнес-логіки та інтеграційні тести.
6. **OpenAPI/Swagger документація** — Автоматична документація REST-ендпоінтів.
7. **Обробка помилок** — Глобальні ExceptionHandler, коректні HTTP-відповіді.

Як працюємо над проєктом

Це завдання — **фінальний проєкт**, і ми будемо працювати над ним **протягом двох тижнів**:

✓ **Після 14 теми**

- Налаштовуєш Spring Boot + PostgreSQL
- Створюєш сутності та JPA-зв'язки
- Створюєш Repository Layer
- Реалізуєш CRUD-операції через Service + Controller
- Налаштовуєш Swagger

✓ **Після 15 теми**

- Додаєш Enrollment, оплати й оцінки
- Реалізуєш звіти та фільтрацію
- Пишеш тести
- Перевіряєш увесь функціонал
- Готуєш GitHub-репозиторій з README

⌚ Тобто часу достатньо, щоб зробити якісно, головне — плануй етапи й не відкладай все на останню хвилину.

Технічні Вимоги та Стек

1. **Базові Технології:** Java 21+, Maven/Gradle.

- 2. Фреймворки:** Spring Boot 3+, Spring Data JPA, Spring Web.
- 3. База Даних:** В якості сховища даних необхідно використовувати реляційну базу даних (наприклад, PostgreSQL).
- 4. Сховище:** Дані повинні зберігатися у базі даних. Зберігання даних у пам'яті програми (in-memory storage) **не допускається**.
- 5. Архітектура:** Обов'язкове використання багат шарової архітектури:

Controller Layer (REST API endpoints).

Service Layer (Бізнес-логіка, GPA-обчислення, фільтрація).

Repository Layer (Взаємодія з БД через Spring Data JPA).

- 6. Взаємодія:** Застосунок повинен надавати тільки **REST API**. Розробка фронтенду (UI) **не вимагається**.

Функціональні Вимоги (REST API)

Усі наступні функціональні вимоги мають бути реалізовані як **RESTful ендпоінти** відповідно до принципів багат шарової архітектури. Проєкт має забезпечити повний цикл управління наступними ключовими сутностями: **Студенти (Student)**, **Викладачі (Teacher)**, **Курси (Course)** та **Зарахування (Enrollment)**.

А. Модель Даних та Сутності (JPA)

Для реалізації функціоналу необхідно розробити наступні JPA-сутності та зв'язки між ними. Всі сутності повинні мати первинний ключ (**id**).

Таблиця 1: Основні Сутності та Поля

Сутність	Основні Поля	Enum/Тип	Примітки
Student (Студент)	firstName, lastName, email, enrollmentYear, status	StudentStatus	Статус: ACTIVE, ON_LEAVE, EXPELLED, GRADUATED.
Teacher (Викладач)	firstName, lastName, email, dateOfBirth, position	TeacherPosition	Посада: ASSISTANT, LECTURER, PROFESSOR.
Course (Курс)	name, credits, description	Integer	Кількість кредитів (ECTS).
Enrollment (Зарахування)	semester, year, paid, grade	Grade, Boolean	Оцінка: A, B, C, D, F, NA.

Таблиця 2: Зв'язки Між Сутностями

Зв'язок	Сутність 1	Тип Зв'язку	Сутність 2	Примітки
1	Teacher	One-to-Many	Course	Один викладач може викладати багато курсів.
2	Student	One-to-Many	Enrollment	Один студент має багато записів про зарахування.
3	Course	One-to-Many	Enrollment	Один курс має багато записів про зарахування (від різних студентів).
4	Enrollment	Many-to-One	Student & Course	Сутність Enrollment є зв'язною (Junction Table).

В. Модулі Сутностей та Основні Операції (CRUD)

Необхідно реалізувати повноцінні **CRUD-операції (Create, Read, Update, Delete)** для кожної основної сутності з дотриманням REST-конвенцій (використання правильних HTTP-методів та статусів).

Об'єкт	Сутності та Enum'и	Метод	URL-шлях	Опис
Студент	Використовувати <code>StudentStatus</code> enum.	POST	<code>/api/students</code>	Додати нового студента.
		GET	<code>/api/students</code>	Отримати список усіх студентів (з пагінацією/сортуванням).
		GET	<code>/api/students/{id}</code>	Отримати дані конкретного студента.
		PUT	<code>/api/students/{id}</code>	Оновити дані студента.
		DELETE	<code>/api/students/{id}</code>	Видалити студента.
Викладач	Використовувати <code>TeacherPosition</code> enum.	CRUD-операції	<code>/api/teachers</code>	Аналогічно студентам.
Курс		CRUD-операції	<code>/api/courses</code>	Аналогічно студентам.

С. Управління Зарахуванням та Оцінками (Enrollment)

Для управління процесом навчання необхідно реалізувати сутність `Enrollment` та наступні операції:

Модуль	Enum'и та Інтерфейси	Метод	URL-шлях	Опис
Зарахування	Використовувати <code>Grade</code> enum.	POST	<code>/api/enrollments</code>	Створити нове зарахування (зв'язати <code>studentId</code> + <code>courseId</code> + <code>semester</code>). Початково: <code>grade = NA</code> , <code>paid=false</code> .
Оцінка		PUT	<code>/api/enrollments/{id}/grade</code>	Поставити оцінку.
Оплата	Реалізувати інтерфейс <code>Payable</code> для сутності <code>Enrollment</code> .	PUT	<code>/api/enrollments/{id}/paid</code>	Позначити оплату (змінити <code>paid=true</code>).
Транскрипт	Обчислення GPA	GET	<code>/api/students/{id}/transcript</code>	Отримати детальний транскрипт студента з підрахунком GPA.

D. Звіти та Фільтрація (Spring Data)

Необхідно реалізувати наступні складніші запити, використовуючи можливості **Spring Data JPA** (Query Methods, JPQL або Criteria API).

1. Фільтрація студентів:

За статусом (`/api/students?status=...`)

За роком навчання (`/api/students?year=...`)

2. Пошук: Пошук студента за частиною ПІБ або email.

3. Неоплачені курси: Список студентів, які мають хоча б один неоплачений курс (`paid=false`).

4. Середній GPA: Обчислення середнього GPA по конкретному курсу або семестру.

5. Топ студентів: Список топ-N студентів за найвищим GPA.

6. Фільтр курсів: Фільтр курсів за викладачем або кількістю кредитів.

Нефункціональні Вимоги

1. REST Конвенції — Чітке дотримання принципів REST: використання множини в URL (e.g., `/students`), коректні HTTP-статуси (200 OK, 201 Created, 400 Bad Request, 404 Not Found), обробка параметрів запити.

2. **DTOs (Data Transfer Objects)** — Обов'язкове використання DTO для передачі даних між Controller та клієнтом (уникайте прямої передачі JPA-сутностей).

3. **Документація (Swagger)** — **ОБОВ'ЯЗКОВО:** Інтеграція **Springdoc-OpenAPI** для автоматичної генерації документації API (Swagger UI). Всі ендпоінти мають бути коректно описані.

4. **Обробка помилок** — Централізована обробка винятків за допомогою `@ControllerAdvice` для повернення зрозумілих HTTP-помилки (наприклад, 400 Bad Request при невалідному вводі даних або 404 Not Found при відсутності сутності).

5. **Тестування** — **ОБОВ'ЯЗКОВО:** Проєкт повинен містити покриття тестами.

Структура Проєкту

Рекомендована структура проєкту для забезпечення багатoshарової архітектури:

```
src/main/java/ua/university/sms/  
├── controller/  
│   ├── StudentController.java    // REST endpoints  
│   └── ...  
├── service/  
│   ├── StudentService.java      // Business logic, GPA  
│   │   calculation  
│   └── ...  
├── repository/  
│   ├── StudentRepository.java    // Spring Data JPA  
│   │   interfaces  
│   └── ...  
├── model/  
│   ├── entity/                  // JPA entities  
│   │   (Student, Course, Enrollment, ...)  
│   └── dto/                      // DTOs  
│   │   (StudentRequest, StudentResponse, ...)  
└── exception/                   // Custom exceptions  
    and @ControllerAdvice
```

Перевір себе перед здачею

Перед здачею перевір, що:

- ✓ CRUD для Student, Teacher, Course працює
- ✓ Enrollment створюється, ставиться оцінка й оплата
- ✓ GPA рахується правильно
- ✓ Є фільтрація та пошук
- ✓ DTO використовуються всюди
- ✓ Всі ендпоінти доступні в Swagger UI
- ✓ Є хоча б 8–10 тестів
- ✓ Код структурований за шарами
- ✓ Репозиторій на GitHub + README
- ✓ Все запускається без помилок

- ✓ HTTP-статуси коректні
- ✓ Обробка винятків працює

! Якщо чогось із цього немає — проєкт треба буде доопрацьовувати.

Як буде оцінюватися проєкт (40 балів)

Проєкт оцінюється за наступною схемою, де акцент зроблено на застосуванні Spring-екосистеми та якості розробки:

Критерій	Балів	Опис
1. Архітектура та Spring-екосистема	10	Правильна багатoshарова архітектура (Controller, Service, Repository). Коректне використання DI, конфігурації та анотацій Spring.
2. Функціональність та Persistence	10	Повна реалізація всіх CRUD-операцій, логіки зарахування, GPA, звітів. Безпомилкова робота з Spring Data JPA та базою даних.
3. API Design та Документація	8	Дотримання REST-конвенцій. Використання DTO. Повна та коректна документація API через Swagger/Springdoc-OpenAPI .
4. Тестування (JUnit/Mockito)	8	Наявність та якість тестів: Unit-тести для бізнес-логіки (Service Layer). Інтеграційні тести (Repository та Controller Layer, наприклад, із використанням <code>@WebMvcTest</code> або <code>@SpringBootTest</code>).
5. Якість Коду та Git	4	Читабельність, інкапсуляція, коректна обробка винятків, правильне оформлення Git-репозиторію (гілки, коміти, <code>README.md</code>).
РАЗОМ	40 балів	

Поради

Перш ніж ти перейдеш до завершального етапу роботи, маю для тебе кілька порад. Вони допоможуть уникнути типових помилок, раціонально розподілити час і зробити проєкт дійсно якісним і зрозумілим.

Сприймай їх як чекліст від ментора, який пройшов цей шлях багато разів 😊

Почни з JPA: перш ніж писати контролери, налаштуй JPA та переконайся, що ти можеш зберігати та отримувати прості сутності (наприклад, `Student`) з бази даних.

Сфокусуйся на REST: визнач, які DTO тобі потрібні для вхідних даних (Requests) та вихідних (Responses).

Тестуй одразу: не відкладай написання тестів на кінець. Напиши простий інтеграційний тест для контролера після того, як реалізуєш перший CRUD.

Використовуйте Spring Data: для фільтрації та пошуку максимально використовуй можливості Spring Data JPA (наприклад, `findByStatusAndYear()`) замість ручної реалізації логіки.

Фінальний проєкт готовий — що далі?

Перевір, що все готово й виглядає зрозуміло.

Запуш свій код у GitHub та скопіюй **посилання** із твоїм виконаним завданням.

Відкрий доступ до файлу, щоб ментор міг його переглянути.

Прикріпи посилання у віконечко **Домашнього завдання** на платформі.

Натисни **«Відправити»** до дедлайну.